

Semáforos

Apunte para el examen teórico: qué es un semáforo, cómo reemplaza al busy waiting, y los problemas clásicos que se resuelven con él — hasta llegar a passing the baton y la asignación de recursos.

MEMORIA COMPARTIDA – RECORRIDO DE LA MATERIA

Locks y Barreras



Semáforos



Monitores

● Fundamentos

● Exclusión mutua

● Señalización

● Buffers limitados

● Deadlock y filósofos

● Lectores/escritores

● Passing the baton

● Asignación (SJN)

MÓDULO 1

01

Semáforos – fundamentos

Los protocolos de busy waiting (Teoría 2) eran complejos, difíciles de verificar y desperdiciaban procesador. El semáforo es la primera herramienta de más alto nivel para resolver eso.

¿Qué es un semáforo?

Descrito por Dijkstra en 1968. Es un tipo de dato abstracto (o un objeto) con solo dos operaciones, ambas **atómicas**: P y V. Su valor interno es siempre un entero **no negativo**.

- **V** señala la ocurrencia de un evento — incrementa el semáforo.
- **P** demora un proceso hasta que ocurra un evento — decrementa el semáforo.

Analogía: la sincronización del tránsito para evitar colisiones. Sirve tanto para proteger secciones críticas como para implementar sincronización por condición.

Declaración y tipos

```
sem s; // x NO – todo semáforo debe inicializarse en la
declaración
sem mutex = 1;
sem fork[5] = ([5] 1);
```

General (counting)

```
P(s): <await(s>0) s=s-1;>
V(s): <s=s+1;>
```

Binario

```
P(b): <await(b>0) b=b-1;>
V(b): <b=b+1;>
```

Si la demora por P se implementa sobre una cola, las operaciones son *fair* — pero en esta materia no se puede suponer ese tipo de implementación.

MÓDULO 2

02

Exclusión mutua con semáforos

El semáforo binario resuelve exclusión mutua de forma directa. Se llega a él transformando, paso a paso, el lock de grano grueso de la Teoría 2.

```
bool lock=false; // await no atómico (T2)
<await(not lock) lock=true;>
  ↓ cambio de variable
int free=1;
<await(free>0) free=free-1;> <free=free+1;>
  ↓ free ya se comporta como un semáforo
sem free=1;
process SC[i=1 to n] { while(true) {
  P(free);
  sección crítica;
  V(free);
  sección no crítica;
} }
```

Es más simple que cualquier solución busy-waiting: no hace falta elegir entre spin locks, tie-breaker, ticket o bakery — el propio semáforo garantiza la exclusión.

¿Y SI INICIALIZO $FREE = 0$?

Todo proceso que llegue a $P(free)$ encuentra el semáforo en 0 y queda demorado para siempre — nadie puede entrar nunca a la SC. La inicialización en 1 es lo que representa "el recurso está libre".

03

MÓDULO 3

Señalización de eventos: barreras y SBS

Además de proteger secciones críticas, un semáforo puede usarse para avisar "esto ya pasó" — la base de las barreras y de los semáforos binarios divididos.

| Barrera para dos procesos

Idea: un semáforo por cada *flag*, inicializado en 0. Un proceso señala su llegada con V y espera la del otro con P.

```
sem llegal=0, llega2=0;
process Worker1 { ...; V(llegal); P(llega2); ... }
process Worker2 { ...; V(llega2); P(llegal); ... }
```

Se puede extender a n procesos con una *butterfly barrier* o con un coordinador central.

¿QUÉ PASA SI PRIMERO HACEN P Y LUEGO V?

Deadlock: cada proceso queda esperando (P) la señal del otro antes de que ninguno haya señalado (V) todavía. El orden V-antes-que-P del que espera es lo que hace que la barrera funcione.

| Semáforo Binario Dividido (SBS)

Los semáforos binarios $b_1, \dots, b_n$ forman un SBS si vale el invariante global **SPLIT**: $0 \leq b_1 + \dots + b_n \leq 1$ — se los puede pensar como un único semáforo binario repartido en n . Son la base para implementar exclusión mutua donde la ejecución empieza con un P sobre un semáforo y termina con un V sobre *otro*; entre el P y el V hay exclusión mutua garantizada.

```
typeT buf; sem vacio=1, lleno=0;
process Productor[i=1 to M] { while(true) {
  producir datos;
  P(vacio); buf=datos; V(lleno); // depositar
} }
process Consumidor[j=1 to N] { while(true) {
  P(lleno); resultado=buf; V(vacio); // retirar
  consumir resultado;
} }
```

vacio y lleno juntos forman un SBS: buffer unitario, exactamente uno de los dos vale 1 en cada momento.

MÓDULO 4

04

Buffers limitados – contadores de recursos

Cuando el recurso tiene varias unidades (no solo una casilla), el semáforo cuenta cuántas hay libres — adecuado cuando los procesos compiten por recursos de múltiples unidades.

1 productor, 1 consumidor

```
typeT buf[n]; int ocupado=0, libre=0;
sem vacio=n, lleno=0;
process Productor { while(true) {
    producir datos;
    P(vacio); buf[libre]=datos; libre=(libre+1) mod n; V(lleno);
} }
process Consumidor { while(true) {
    P(lleno); resultado=buf[ocupado]; ocupado=(ocupado+1) mod n; V(vacio);
    consumir resultado;
} }
```

vacio cuenta lugares libres, lleno los ocupados. Con un único productor y un único consumidor, depositar y retirar pueden asumirse atómicas.

Múltiples productores y/o consumidores

Ahí depositar y retirar dejan de ser atómicas por sí solas: se vuelven secciones críticas propias, y necesitan su exclusión mutua adicional — si no se protege cada slot, se podría retirar dos veces el mismo dato o perder datos por sobreescritura.

```
sem vacio=n, lleno=0, mutexD=1, mutexR=1;
process Productor[i=1..M] { while(true) {
    producir datos; P(vacio);
    P(mutexD); buf[libre]=datos; libre=(libre+1) mod n; V(mutexD);
    V(lleno);
} }
process Consumidor[i=1..N] { while(true) {
    P(lleno);
    P(mutexR); resultado=buf[ocupado]; ocupado=(ocupado+1) mod n; V(mutexR);
    V(vacio); consumir resultado;
} }
```

Varios recursos: deadlock y filósofos

Cuando un proceso necesita más de un recurso a la vez, adquirir sus locks en el orden equivocado puede generar un deadlock — aunque cada recurso individual esté bien protegido.

Varios procesos, varios recursos

Un proceso debe adquirir los locks de *todos* los recursos que necesita. Si varios procesos compiten por conjuntos de recursos superpuestos, puede caerse en deadlock. Ejemplo clásico: cada $P[i]$ necesita $R[i]$ y $R[(i+1) \bmod n]$ — si todos toman su primer lock a la vez, quedan esperando en círculo el segundo (espera cíclica, ver las 4 condiciones de deadlock de la Teoría 1).

Problema de los filósofos – exclusión mutua selectiva

Procesos que compiten por el acceso a conjuntos superpuestos de variables compartidas. Cada tenedor es una SC (a lo sumo un filósofo a la vez), representable con un arreglo de semáforos: levantar tenedor = P, bajar tenedor = V. Cada filósofo necesita el tenedor izquierdo y el derecho.

```
sem tenedores[5] = {1,1,1,1,1};
process Filosofo[i=0..3] { while(true) {
  P(tenedor[i]); P(tenedor[i+1]);
  comer;
  V(tenedor[i]); V(tenedor[i+1]);
} }
process Filosofo[4] { while(true) {
  P(tenedor[0]); P(tenedor[4]); // orden invertido
  comer;
  V(tenedor[0]); V(tenedor[4]);
} }
```

¿QUÉ PASA SI TODOS HACEN EXACTAMENTE LO MISMO?

Deadlock: los 5 toman su tenedor izquierdo al mismo tiempo, y ninguno consigue el derecho — espera cíclica. Por eso el filósofo 4 pide sus tenedores en el orden inverso al resto: rompe la simetría del ciclo y evita el deadlock.

Lectores y escritores

Dos clases de procesos comparten una base de datos: los escritores necesitan acceso exclusivo (evitar interferencia), los lectores pueden convivir entre sí si no hay ningún escritor activo. Es exclusión mutua *selectiva*: compiten clases de procesos, no procesos individuales.

| Enfoque 1 – como exclusión mutua simple

```
sem rw=1;
process Lector[i=1..M] { ...; P(rw); lee la BD; V(rw); }
process Escritor[j=1..N] { ...; P(rw); escribe la BD; V(rw); }
```

Correcto, pero no hay concurrencia entre lectores — un solo semáforo trata a todos por igual.

| Enfoque mejorado – contador de lectores activos

Solo el **primer** lector necesita tomar rw (bloquea a los escritores); solo el **último** lector debe liberarlo. Un contador nr, protegido por su propio mutexR, lleva la cuenta.

```
int nr=0; sem rw=1, mutexR=1;
process Lector[i=1..M] { while(true) {
  P(mutexR); nr=nr+1; if(nr==1) P(rw); V(mutexR);
  lee la BD;
  P(mutexR); nr=nr-1; if(nr==0) V(rw); V(mutexR);
} }
process Escritor[j=1..N] { while(true) { P(rw); escribe la BD; V(rw); } }
```

Da preferencia a los lectores → no es fair: un escritor puede quedar esperando indefinidamente si siempre hay algún lector activo (inanición).

MÓDULO 7

07

Técnica Passing the Baton

A veces un solo semáforo no alcanza para implementar un await arbitrario. Por eso hace falta una técnica general.

| Por qué no alcanza un semáforo simple

Con sincronización por condición, un escritor necesita esperar $nr==0$ and $nw==0$, mientras un lector solo necesita $nw==0$ — las guardas se superponen parcialmente, y ningún semáforo puede discriminar entre esas dos condiciones distintas con un simple P/V.

| La técnica

Baton (testimonio/token) = permiso para ejecutar. Quien está dentro de la SC lo tiene. Al llegar a un SIGNAL (salir de la SC), el proceso *pasa* el baton a otro proceso que lo esté esperando; si nadie espera, el baton se libera para el próximo que intente entrar.

```
F1: P(e); Si; SIGNAL; // {Si}

F2: P(e);
    if (not Bj) { dj=dj+1; V(e); P(bj); }
    Sj; SIGNAL; // {await(Bj) Sj}

SIGNAL: if (B1 and d1>0) {d1=d1-1; V(b1)}
        [] ...
        [] (Bn and dn>0) {dn=dn-1; V(bn)}
        [] else V(e);
        fi
```

e es un semáforo (inicialmente 1) que controla la entrada; cada guarda distinta Bj tiene su propio semáforo bj y su contador de demorados dj (ambos en 0). e junto con los bj forman un SBS.

| Aplicación: lectores/escritores más justo

Contando también los procesos demorados de cada clase (dr, dw), cada SIGNAL_i decide a quién pasarle el baton — típicamente: primero a un lector demorado si hay, si no a un escritor demorado, si no se libera e. Así se puede controlar con precisión el orden en que los procesos son despertados, en vez de que una clase monopolice el acceso como en el enfoque del módulo anterior.

MÓDULO 8

08

Alocación de recursos y scheduling

El problema general: decidir cuándo un proceso puede acceder a un recurso compartido de múltiples unidades, cada una libre o en uso — con request/release implementados vía passing the baton.

```
request(parámetros): P(e);
    if (request no puede ser satisfecho) DELAY;
    tomar unidades; SIGNAL;
release(parámetros): P(e);
    retornar unidades; SIGNAL;
```

| Caso: Shortest-Job-Next (SJN)

Un recurso de **una sola unidad**. request(tiempo,id): si está libre, se asigna de inmediato; si no, el proceso se demora. release(): al liberarse, se asigna al

demorado con menor tiempo (empate → el que esperó más).

SJN minimiza el tiempo promedio de ejecución, pero es **unfair**: un trabajo largo puede sufrir inanición si siempre llegan trabajos más cortos. Se mejora con *aging* (dar preferencia a quien esperó mucho).

```
bool libre=true; Pares=set of(int,int)=∅; sem e=1, b[n]=([n] 0);

request(tiempo,id): P(e);
    if (!libre) { insertar(tiempo,id) en Pares; V(e); P(b[id]); }
    libre=false; V(e);

release(): P(e);
    libre=true;
    if (Pares≠∅) { remover el primer par (tiempo,id) de Pares; V(b[id]); }
    else V(e);
```

Cada $b[id]$ es un **semáforo privado**: solo el proceso id ejecuta P sobre él — útil para señalar a un proceso individual en vez de a cualquiera que espere.

CIERRE

De acá en adelante

Los semáforos resuelven exclusión mutua y sincronización por condición de forma más simple que el busy waiting, pero exigen disciplina: mal usados (orden de P/V invertido, falta de un mutex extra, mala inicialización) generan exactamente los mismos problemas — deadlock, inanición. La Teoría 4 (Monitores) encapsula esta disciplina dentro del lenguaje, para que el programador no tenga que garantizarla a mano.